

Heuristic Evaluation - BugBoard

Scopo: guidare una heuristic evaluation uniforme di BugBoard sui flussi reali dell'applicazione.

Modalità di lavoro: compilazione individuale, flow-by-flow, usando sempre le 10 euristiche di Nielsen.

Istruzioni Generali

- Compilare questo documento individualmente, senza confronto con altri valutatori durante la sessione.
- Per ogni flow eseguire il task completo e compilare tutte le righe della tabella.
- Se non emerge alcun problema per una riga, scrivere **Nessun problema rilevato**.
- Se un flow non è accessibile per il ruolo usato, scrivere **Non applicabile per questo ruolo**.
- Registrare solo evidenze osservabili, non giudizi vaghi.

Campi Da Usare Nelle Tabelle

Campo	Indicazione
Esito	OK / Problema / Non verificabile / Non applicabile
Evidenza / problema	Comportamento osservato, punto di attrito, errore o ambiguità
Severità	0, 1, 2, 3, 4
Nota	Conseguenza per l'utente o proposta sintetica di miglioramento

Scala Di Severità

Valore	Significato
0	Nessun problema di usabilità rilevato
1	Problema lieve o cosmetico, non incide realmente sul task
2	Problema moderato, rallenta o genera incertezza
3	Problema serio, compromette il task o genera errori frequenti
4	Problema critico, blocca il task o genera alto rischio di fallimento

Ordine Dei Flow

1. Login
2. Recupero password
3. Home progetti e apertura progetto
4. Backlog issue del progetto

- 5. Creazione issue
- 6. Dettaglio issue, activity, commenti e allegati
- 7. Modifica issue e gestione assegnazione
- 8. Notifiche globali
- 9. Impostazioni profilo
- 10. Creazione progetto e gestione team (admin)
- 11. Modifica o cancellazione progetto (admin)
- 12. Amministrazione utenti (admin)

Flow 1. Login

Ruolo: developer o admin

Obiettivo utente: accedere all'area autenticata

Schermate: /login

Passi: aprire login, provare submit vuoto, provare credenziali errate, provare credenziali corrette, verificare redirect a /projects.

Focus osservativo: campi, messaggi di errore, feedback di submit, comprensibilità dell'esito.

N	Euristica	Cosa controllare in questo flow	Esito	Evidenza / problema	Severità	Nota
1	Visibilità dello stato del sistema	È chiaro quando il login è in corso e quando termina?				
2	Corrispondenza tra sistema e mondo reale	Etichette e messaggi sono comprensibili per uno studente/utente gestionale?				
3	Controllo e libertà dell'utente	L'utente può correggere facilmente input errati senza perdere orientamento?				
4	Coerenza e standard	Form, pulsanti e feedback seguono convenzioni note?				
5	Prevenzione degli errori	Il sistema previene invii ambigui o incompleti?				
6	Riconoscimento invece di memorizzazione	L'utente capisce subito cosa inserire senza dover interpretare troppo?				
7	Flessibilità ed efficienza d'uso	Il flow è rapido anche per utenti abituali?				
8	Design estetico e minimalista	La schermata mostra solo ciò che serve al login?				
9	Aiutare a riconoscere, diagnosticare e recuperare dagli errori	Gli errori spiegano bene cosa fare dopo?				
10	Aiuto e documentazione	È chiaro dove andare se non si ricorda la password?				

Flow 2. Recupero Password

Ruolo: developer o admin

Obiettivo utente: recuperare l'accesso senza login

Schermate: /forgot-password, /forgot-password/verify

Passi: aprire flow, inserire email, procedere allo step OTP, inserire codice, inserire nuova password, verificare ritorno al login.

Focus osservativo: chiarezza processo a 2 step, errori OTP/password, contesto email, recuperabilità.

N	Euristica	Cosa controllare in questo flow	Esito	Evidenza / problema	Severità	Nota
1	Visibilità dello stato del sistema	Ogni passaggio del reset comunica chiaramente esito e avanzamento?				
2	Corrispondenza tra sistema e mondo reale	Il flow OTP è spiegato con linguaggio comprensibile?				
3	Controllo e libertà dell'utente	L'utente può tornare indietro o ricominciare senza confusione?				
4	Coerenza e standard	I due step del recupero sono coerenti come struttura e feedback?				
5	Prevenzione degli errori	I campi riducono errori su email, OTP e nuova password?				
6	Riconoscimento invece di memorizzazione	Il sistema riduce la necessità di ricordare cosa fare nel secondo step?				
7	Flessibilità ed efficienza d'uso	Il flow è lineare e non ridondante?				
8	Design estetico e minimalista	Le schermate evitano rumore visivo e distrazioni?				
9	Aiutare a riconoscere, diagnosticare e recuperare dagli errori	Se OTP o password non vanno bene, il problema è chiaro è recuperabile?				
10	Aiuto e documentazione	Il sistema spiega cosa fare se manca il contesto email o il codice non funziona?				

Flow 3. Home Progetti E Apertura Progetto

Ruolo: developer o admin

Obiettivo utente: orientarsi e aprire un progetto

Schermate: /projects

Passi: entrare nella home, osservare lista progetti, usare la ricerca, aprire un progetto.

Focus osservativo: orientamento iniziale, ricerca, leggibilità card progetto, comprensione della home reale.

N	Euristica	Cosa controllare in questo flow	Esito	Evidenza / problema	Severità	Nota
1	Visibilità dello stato del sistema	È chiaro se i progetti sono in caricamento, assenti o filtrati?				
2	Corrispondenza tra sistema e mondo reale	La home comunica chiaramente che qui si scelgono i progetti di lavoro?				
3	Controllo e libertà dell'utente	L'utente può cercare e cambiare progetto facilmente?				
4	Coerenza e standard	Ricerca, card e CTA seguono pattern coerenti?				
5	Prevenzione degli errori	La UI evita click ambigui o incomprensioni sulla selezione progetto?				
6	Riconoscimento invece di memorizzazione	I progetti sono riconoscibili a colpo d'occhio?				
7	Flessibilità ed efficienza d'uso	La ricerca aiuta davvero a trovare rapidamente un progetto?				
8	Design estetico e minimalista	La schermata è focalizzata sui progetti senza elementi superflui?				
9	Aiutare a riconoscere, diagnosticare e recuperare dagli errori	Empty state o error state aiutano a capire cosa fare?				
10	Aiuto e documentazione	Le istruzioni iniziali bastano a capire l'azione da compiere?				

Flow 4. Backlog Issue Del Progetto

Ruolo: developer o admin

Obiettivo utente: trovare e aprire una issue rilevante

Schermate: /projects/:projectId/issues

Passi: aprire progetto, usare ricerca, filtri, ordinamento, filtro Assigned to you se developer, aprire una issue.

Focus osservativo: filtri, card issue, orientamento nel progetto, efficacia nella ricerca dei record.

N	Euristica	Cosa controllare in questo flow	Esito	Evidenza / problema	Severità	Nota
1	Visibilità dello stato del sistema	È evidente quando la lista è caricata, aggiornata o vuota?				
2	Corrispondenza tra sistema e mondo reale	Stati, priorità e tipi sono espressi in modo comprensibile?				
3	Controllo e libertà dell'utente	Ricerca e filtri si possono applicare e cambiare facilmente?				
4	Coerenza e standard	I controlli di filtro sono coerenti con altre aree dell'app?				
5	Prevenzione degli errori	La UI evita filtri poco chiari o combinazioni non interpretabili?				
6	Riconoscimento invece di memorizzazione	È sempre chiaro quale progetto si sta consultando e quali filtri sono attivi?				
7	Flessibilità ed efficienza d'uso	Il backlog permette di individuare rapidamente l'issue desiderata?				
8	Design estetico e minimalista	Le issue mostrano informazioni utili senza sovraccaricare la lista?				
9	Aiutare a riconoscere, diagnosticare e recuperare dagli errori	L'utente capisce come uscire da una ricerca senza risultati?				
10	Aiuto e documentazione	Le etichette dei filtri bastano a capire come usarli?				

Flow 5. Creazione Issue

- Ruolo:** developer o admin
- Obiettivo utente:** segnalare una nuova issue
- Schermate:** backlog issue + modal Report New Issue
- Passi:** aprire modal, compilare campi, aggiungere allegati, inviare, verificare feedback finale.
- Focus osservativo:** chiarezza form, validazioni, limiti allegati, conferma di creazione.

N	Euristica	Cosa controllare in questo flow	Esito	Evidenza / problema	Severità	Nota
1	Visibilità dello stato del sistema	Il sistema comunica bene caricamento, successo o warning?				
2	Corrispondenza tra sistema e mondo reale	I campi del form corrispondono al modo in cui un utente pensa una segnalazione?				
3	Controllo e libertà dell'utente	È facile chiudere il modal o correggere i dati prima dell'invio?				
4	Coerenza e standard	Campi, contatori, select e pulsanti sono coerenti con il resto dell'app?				
5	Prevenzione degli errori	Obbligatorietà, vincoli e limiti allegati sono chiari prima dell'errore?				
6	Riconoscimento invece di memorizzazione	Il form rende visibili limiti, opzioni e stato di compilazione?				
7	Flessibilità ed efficienza d'uso	Il flow consente di creare una issue in modo rapido ma completo?				
8	Design estetico e minimalista	Il modal evita complessità non necessaria?				
9	Aiutare a riconoscere, diagnosticare e recuperare dagli errori	In caso di submit fallito o warning parziale il messaggio è chiaro?				
10	Aiuto e documentazione	Le info su allegati e campi bastano a guidare l'utente?				

Flow 6. Dettaglio Issue, Activity, Commenti E Allegati

Ruolo: developer o admin

Obiettivo utente: capire lo stato della issue e collaborare tramite timeline

Schermate: /projects/:projectId/issues/:issueId

Passi: aprire issue, leggere sidebar e timeline, aggiungere commento se consentito, allegare file, usare preview/download, osservare nuovi messaggi.

Focus osservativo: leggibilità del dettaglio, separazione sidebar/timeline, inserimento commenti, blocchi dovuti a stato o ruolo.

N	Euristica	Cosa controllare in questo flow	Esito	Evidenza / problema	Severità	Nota
1	Visibilità dello stato del sistema	Stato issue, nuovi messaggi, invio commento e caricamento sono sempre visibili?				
2	Corrispondenza tra sistema e mondo reale	Timeline, commenti e dettagli usano termini e struttura naturali?				
3	Controllo e libertà dell'utente	L'utente può gestire commento, allegati, preview e ritorno senza perdere orientamento?				
4	Coerenza e standard	Sidebar, timeline e preview allegati seguono pattern coerenti?				
5	Prevenzione degli errori	Il sistema previene invii vuoti, allegati non validi o azioni non consentite?				
6	Riconoscimento invece di memorizzazione	È immediato capire stato, assegnari, reporter, tag e storico?				
7	Flessibilità ed efficienza d'uso	Il dettaglio supporta bene sia lettura rapida sia collaborazione frequente?				
8	Design estetico e minimalista	Il layout aiuta a distinguere tra metadati e attività senza confondere?				
9	Aiutare a riconoscere, diagnosticare e recuperare dagli errori	Se il commento o gli allegati falliscono, il problema è interpretabile e recuperabile?				
10	Aiuto e documentazione	Il sistema spiega chiaramente i limiti degli allegati e le ragioni per cui l'utente potrebbe non poter inserire un commento?				

Flow 7. Modifica Issue E Gestione Assegnazione

Ruolo: developer per edit consentito, admin per gestione completa

Obiettivo utente: aggiornare issue e assegnatari

Schermate: dettaglio issue + modal edit + modal assignees

Passi: aprire edit issue, modificare metadati, salvare, aprire gestione membri, aggiungere/rimuovere assegnatari, salvare.

Focus osservativo: campi modificabili, visibilità dei permessi, chiarezza selezione utenti, feedback finale.

N	Euristica	Cosa controllare in questo flow	Esito	Evidenza / problema	Severità	Nota
1	Visibilità dello stato del sistema	È chiaro quando la modifica è in corso e quando è stata salvata?				
2	Corrispondenza tra sistema e mondo reale	Le azioni di edit e assegnazione riflettono aspettative realistiche?				
3	Controllo e libertà dell'utente	L'utente può uscire o correggere facilmente prima del salvataggio?				
4	Coerenza e standard	Il comportamento dei modal di edit e team è coerente con gli altri modal?				
5	Prevenzione degli errori	La UI previene assegnazioni sbagliate o modifiche involontarie?				
6	Riconoscimento invece di memorizzazione	È chiaro chi è già assegnato, chi no e cosa cambierà?				
7	Flessibilità ed efficienza d'uso	Il flow è efficiente per cambiare rapidamente metadati o assignees?				
8	Design estetico e minimalista	I modal mostrano solo i controlli realmente necessari?				
9	Aiutare a riconoscere, diagnosticare e recuperare dagli errori	Se il salvataggio fallisce, l'utente capisce cosa correggere?				
10	Aiuto e documentazione	I motivi di sola lettura o limitazione di ruolo sono chiari?				

Flow 8. Notifiche Globali

Ruolo: developer o admin

Obiettivo utente: leggere e raggiungere rapidamente eventi rilevanti

Schermate: top nav + dropdown notifiche

Passi: aprire dropdown, leggere una notifica, marcarla come letta, eliminarne una, aprire il target, verificare eventuale target non disponibile.

Focus osservativo: badge, stato letto/non letto, contenuto notifica, navigazione al target.

N	Euristica	Cosa controllare in questo flow	Esito	Evidenza / problema	Severità	Nota
1	Visibilità dello stato del sistema	Badge, stato di lettura e aggiornamenti realtime sono evidenti?				
2	Corrispondenza tra sistema e mondo reale	Titolo e descrizione della notifica fanno capire il tipo di evento?				
3	Controllo e libertà dell'utente	L'utente può aprire, chiudere, leggere o eliminare senza ambiguità?				
4	Coerenza e standard	Le notifiche usano icone, stati e azioni coerenti?				
5	Prevenzione degli errori	È difficile aprire o cancellare una notifica per errore?				
6	Riconoscimento invece di memorizzazione	La notifica da abbastanza contesto senza richiedere memoria dell'evento?				
7	Flessibilità ed efficienza d'uso	Il dropdown consente accesso rapido alle informazioni rilevanti?				
8	Design estetico e minimalista	Il pannello notifiche resta leggibile anche con più elementi?				
9	Aiutare a riconoscere, diagnosticare e recuperare dagli errori	Se il target non è disponibile, il sistema lo comunica in modo utile?				
10	Aiuto e documentazione	L'utente capisce il significato delle azioni disponibili sulle notifiche?				

Flow 9. Impostazioni Profilo

- Ruolo:** developer o admin
- Obiettivo utente:** aggiornare dati personali, avatar e password
- Schermate:** /settings, tab profilo
- Passi:** aprire settings, modificare dati personali, cambiare avatar, cambiare password, salvare.
- Focus osservativo:** chiarezza campi profilo, crop avatar, cambio password, errori campo-specifici.

N	Euristica	Cosa controllare in questo flow	Esito	Evidenza / problema	Severità	Nota
1	Visibilità dello stato del sistema	Upload avatar, salvataggio profilo e successo finale sono visibili?				
2	Corrispondenza tra sistema e mondo reale	Le sezioni profilo e password sono comprensibili per l'utente?				
3	Controllo e libertà dell'utente	L'utente può annullare, uscire o correggere prima del salvataggio?				
4	Coerenza e standard	I pattern di form e feedback sono coerenti con il resto dell'app?				
5	Prevenzione degli errori	Le validazioni aiutano a prevenire email o password non valide?				
6	Riconoscimento invece di memorizzazione	I campi e i vincoli sono abbastanza visibili da non dover essere ricordati?				
7	Flessibilità ed efficienza d'uso	Il flow supporta aggiornamenti rapidi per utenti ricorrenti?				
8	Design estetico e minimalista	La schermata mantiene focus sul profilo senza dispersioni?				
9	Aiutare a riconoscere, diagnosticare e recuperare dagli errori	Gli errori indicano bene il campo e la correzione necessaria?				
10	Aiuto e documentazione	Il collegamento a recupero password o supporto è abbastanza chiaro?				

Flow 10. Creazione Progetto E Gestione Team

Ruolo: admin

Obiettivo utente: creare un progetto e definire il team iniziale

Schermate: /projects + modal create project a 2 step

Passi: aprire create project, compilare dettagli, passare allo step team, cercare utenti, aggiungere/rimuovere membri, creare progetto.

Focus osservativo: chiarezza del flow a step, selezione team, continuita tra step, validazioni.

N	Euristica	Cosa controllare in questo flow	Esito	Evidenza / problema	Severità	Nota
1	Visibilità dello stato del sistema	Il passaggio tra step e il salvataggio finale sono chiari?				
2	Corrispondenza tra sistema e mondo reale	Titolo, descrizione, icona, colore e team riflettono bene il concetto di progetto?				
3	Controllo e libertà dell'utente	L'admin può tornare indietro o uscire senza confusione?				
4	Coerenza e standard	I due step sono coerenti tra loro e con altri flow modal?				
5	Prevenzione degli errori	La UI previene omissioni e selezioni errate dei membri?				
6	Riconoscimento invece di memorizzazione	È sempre chiaro a che punto del flow ci si trova e chi è selezionato?				
7	Flessibilità ed efficienza d'uso	Ricerca, filtri e selezione membri rendono il flow efficiente?				
8	Design estetico e minimalista	Il flow a step resta pulito e focalizzato?				
9	Aiutare a riconoscere, diagnosticare e recuperare dagli errori	In caso di problema, il messaggio aiuta a correggere senza ripartire da zero?				
10	Aiuto e documentazione	Le istruzioni sugli step e sul ruolo dei membri sono sufficienti?				

Flow 11. Modifica O Cancellazione Progetto

- Ruolo:** admin
- Obiettivo utente:** aggiornare o rimuovere un progetto
- Schermate:** backlog progetto + modal edit/delete project
- Passi:** aprire edit project, modificare dati, salvare, aprire delete project, leggere avviso, inserire codice, completare delete.
- Focus osservativo:** azioni distruttive, codice di conferma, feedback post-update e post-delete.

N	Euristica	Cosa controllare in questo flow	Esito	Evidenza / problema	Severità	Nota
1	Visibilità dello stato del sistema	L'admin capisce chiaramente update, delete e relativi esiti?				
2	Corrispondenza tra sistema e mondo reale	L'azione di delete comunica bene conseguenze e irreversibilità?				
3	Controllo e libertà dell'utente	È possibile fermarsi prima dell'azione finale in modo chiaro?				
4	Coerenza e standard	Edit e delete seguono convenzioni coerenti di rischio e conferma?				
5	Prevenzione degli errori	Il sistema protegge da cancellazioni accidentali?				
6	Riconoscimento invece di memorizzazione	Il codice e le istruzioni sono visibili e facili da usare?				
7	Flessibilità ed efficienza d'uso	Il flow non è più complesso del necessario rispetto al rischio dell'azione?				
8	Design estetico e minimalista	Il modal mette in evidenza in modo pulito il rischio dell'azione?				
9	Aiutare a riconoscere, diagnosticare e recuperare dagli errori	Se il codice è errato o la delete fallisce, l'utente capisce come recuperare?				
10	Aiuto e documentazione	Le conseguenze della cancellazione sono spiegate in modo sufficiente?				

Flow 12. Amministrazione Utenti

Ruolo: admin

Obiettivo utente: creare, cercare, modificare, attivare/disattivare utenti

Schermate: /settings tab Add Users, Manage Users, Admin User Edit

Passi: creare un utente, aprire gestione utenti, cercare e filtrare, modificare un utente, cambiare password o avatar, attivare/disattivare un account.

Focus osservativo: chiarezza delle funzioni admin, distinzione tra profilo proprio e altrui, sicurezza percepita delle azioni su account.

N	Euristica	Cosa controllare in questo flow	Esito	Evidenza / problema	Severità	Nota
1	Visibilità dello stato del sistema	Creazione, aggiornamento e cambio stato utente hanno feedback chiari?				
2	Corrispondenza tra sistema e mondo reale	Ruoli, stato attivo/inattivo e azioni admin sono espressi chiaramente?				
3	Controllo e libertà dell'utente	L'admin può uscire, annullare o correggere facilmente?				
4	Coerenza e standard	Add users, manage users ed edit utente sono coerenti tra loro?				
5	Prevenzione degli errori	Le azioni su account sono abbastanza protette da errori operativi?				
6	Riconoscimento invece di memorizzazione	Filtri, stato account e ruolo sono immediatamente leggibili?				
7	Flessibilità ed efficienza d'uso	Ricerca, filtri e paginazione supportano bene gestione rapida di più utenti?				
8	Design estetico e minimalista	Le schermate admin restano ordinate e non sovraccariche?				
9	Aiutare a riconoscere, diagnosticare e recuperare dagli errori	Se un update fallisce, l'errore è localizzato e recuperabile?				
10	Aiuto e documentazione	I limiti sulle azioni admin sono abbastanza chiari?				

Scheda Finale Del Valutatore

Valutatore:

Data:

Ruolo/i usati:

Flow completati:

Flow non eseguibili:

Numero criticità rilevate per severità:

Severità 4:

Severità 3:

Severità 2:

Severità 1:

Severità 0 / nessun problema:

Top 5 criticità:

Aree più problematiche:

Aree con meno problemi osservati:

Dubbi o aspetti non verificabili:

Note finali:
